Nguyễn Dương Bảo Trân

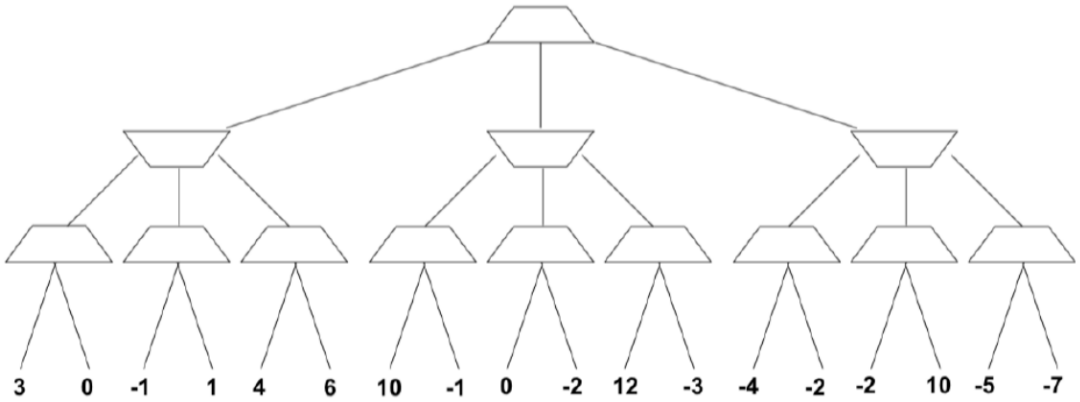
MSSV: 3123410386

BÀI TẬP LÝ THUYẾT

TÌM KIẾM, CSP VÀ GAME TRONG AI

Bài 1. Minimax và Alpha-Beta Pruning

Xét cây trò chơi tổng bằng không trong Hình 1. Nút gốc là MAX, tầng tiếp theo là MIN và tầng kế tiếp là MAX. Hai người chơi đều hành động tối ưu. Các nút con được xét từ trái sang phải.



Hình 1. Cây trò chơi dùng cho Minimax và Alpha-Beta (nguồn: rec3.pdf, trang 1)

a/ Áp dụng Minimax, tính giá trị mỗi nút trung gian trong cây.

Nhánh B:

- B1 max => B1 = max(3,0) = 3
- B2 max => B2 = max(-1,1) = 1
- B3 max => B3 = max(4,6) = 6

B min => B = min(3,1,6) = 1

Nhánh C:

- C1 max => max (10,-1) = 10
- C2 max => max(0,-2) = 0
- C3 max => max(12,-3) = 12

C min => C = min(10,0,12) = 0

Nhánh D:

- D1 max => D1 = max(-4,-2) = -2

- D2 max => D2 = max(-2,10) = 10
- D3 max => D3 = max(-5,-7) = -5

D min => D = min(-2,10,-5) = -5

A max => A = max(B,C,D) = max(1,0,-5) = 1

b/ MAX nên chọn nhánh nào từ nút gốc? Giá trị trò chơi là bao nhiêu?

3 nhánh từ gốc có giá trị B = 1, C = 0, D = -5

Vì A max nên chọn giá trị lớn nhất => A = 1

Do đó MAX chọn nhánh B và giá trị trò chơi tại nút gốc là 1.

c/ Áp dụng Alpha-Beta Pruning theo thứ tự trái sang phải. Ghi α, β ở các nút đã thăm; đánh dấu X các nhánh bị cắt tỉa.

Ban đầu tại A: $\alpha = -\infty;\ \beta = +\infty$

Bước 1: xét nhánh B

B min: $\alpha = -\infty;\ \beta = +\infty$

Xét B1: B1 = max(3,0) = 3

Tại B: $\beta = \min(+\infty, 3) = 3$

Xét B2: B2 = max(-1,1) = 1

Cập nhật tại B: $\beta = \min(3,1) = 1$

Xét B3: $\alpha = -\infty;\ \beta = 1$

Xét lá đầu tiên: 4

Tại nút MAX B3: $\alpha = \max(-\infty, 4) = 4$

Ta có: $\alpha \geq \beta$ nên xảy ra Beta Cutoff.

Vì vậy lá 6 không cần thăm.

Nút B trả về: 1

Quay về A: $\alpha_A = \max(-\infty, 1) = 1$

Bước 2: xét nhánh C

C là MIN và nhận: $\alpha = 1;\ \beta = +\infty$

Xét C1: C1 = max(10,-1) = 10

Tại C: $\beta = \min(+\infty, 10) = 10$

Xét C2: C2 = max(0,-2) = 0

Tại C: $\beta = \min(10,0) = 0$

Lúc này: $\beta \leq \alpha$ nên xảy ra Alpha Cutoff.

Do đó toàn bộ C3 bị cắt gồm hai lá: 12, -3

C trả về: 0

Quay về A: $\alpha_A = \max(1,0) = 1$

Bước 3: xét nhánh D

D là MIN: $\alpha = 1;\ \beta = +\infty$

Xét D1: D1 = max(-4,-2) = -2

Cập nhật tại D: $\beta = \min(+\infty, -2) = -2$

Ta có: $\beta \leq \alpha$ nên xảy ra Alpha Cutoff.

Vì vậy toàn bộ hai nhánh D2, D3 không cần thăm.

Các lá bị cắt là: -2, 10, -5, -7

Alpha-Beta trả về tại D một giá trị đủ để kết luận nhánh này không thể tốt hơn lựa chọn hiện tại của MAX; nếu tính Minimax đầy đủ thì D = -5.

Kết quả cuối cùng

Nút gốc vẫn có: A = 1 và MAX chọn: B

Alpha-Beta không thay đổi kết quả Minimax, chỉ giảm số nút cần duyệt.

d/ Liệt kê các nút lá không cần thăm. So sánh số nút lá được đánh giá bởi Minimax và Alpha-Beta

Tại B3: 6

C3: 12, -3

D2, D3: -2, 10, -5, -7

Tổng số lá bị cắt: 1+2+4 = 7

Số lá minimax đánh giá:

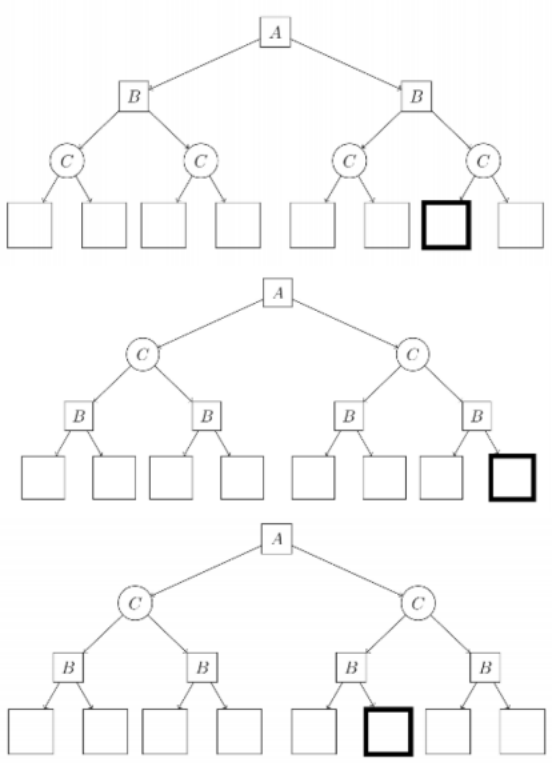
- Cây có 9 nút max ở tầng dưới, mỗi nút có 2 lá: 9x2 = 18. Do đó minimax đầy đủ đánh giá 18 lá.

Số lá alpha-beta đánh giá:

- Alpha-beta cắt 7 lá: 18 -7 = 11 nên chỉ cần đánh giá 11 lá. Số lá giảm được là 7/18 x 100% = 38.9%

Bài 2. Alpha-Beta trong cây có nút ngẫu nhiên (Expinimax)

Xét ba cây trò chơi ở Hình 2. A là nút MIN, B là nút MAX và C là nút ngẫu nhiên (chance node). Các con của C có xác suất bằng nhau. Duyệt từ trái sang phải. Với mỗi cây, hãy gán các giá trị hữu hạn cho các nút lá để có thể cắt tỉa nút được viền đậm, hoặc ghi "Không thể" và giải thích. Không thực hiện cắt tỉa trong trường hợp bằng nhau.



Hình 2. Ba trường hợp cắt tỉa trong cây Expinimax (nguồn: rec3.pdf, trang 3)

a/ Trường hợp 1: điền các giá trị lá (nếu có thể) và chỉ ra điều kiện cắt tỉa.

Chọn các giá trị lá từ trái sang phải để cắt tỉa chẳng hạn: 0,0,0,0,2,2,0,0

Hai chance node của nhánh B trái đều có giá trị 0, nên BL = 0 và tại nút MIN A có beta A = 0.

Ở nhánh B phải, chance node đầu tiên có: C = (2+2)/2 = 2 nên BR ≥ 2 vì 2 > beta A = 0

A chắc chắn không chọn nhánh BR. Do đó chance node còn lại của BR bao gồm nút viền đậm có thể được cắt tỉa.

b/ Trường hợp 2: điền các giá trị lá (nếu có thể) và chỉ ra điều kiện cắt tỉa.

Có thể cắt tỉa. Chọn một bộ giá trị lá: 0,0,0,0,2,2,2,0

Chance node bên trái có giá trị: CL = 0 nên tại A có beta A = 0.

Ở chance node bên phải, nút MAX đầu tiên trả về 2. Ở nút MAX thứ hai, sau khi duyệt lá đầu tiên có giá trị 2, ta biết giá trị của nút MAX này ít nhất là 2. Vì vậy:

CR ≥ ½ (2) + ½ (2) = 2 và: 2 > beta A = 0

nên A sẽ không chọn nhánh phải. Vì vậy lá cuối cùng viền đậm có thể được cắt tỉa.

c/ Trường hợp 3: điền các giá trị lá (nếu có thể) và chỉ ra điều kiện cắt tỉa.

Không thể cắt. Tại thời điểm xét nút viền đậm, mới chỉ đang tính nút MAX đầu tiên của chance node bên phải. Giá trị chance node là: C = (B1+B2)/2 nhưng B2 vẫn chưa được biết. Vì đề không cung cấp giới hạn trên/dưới cho giá trị lá, B2 có thể nhận một giá trị hữu hạn rất nhỏ và làm thay đổi giá trị kỳ vọng của C. Do đó dù giá trị đã quan sát ở B1 lớn đến đâu, ta vẫn chưa thể chắc chắn nhánh chance này sẽ bị nút MIN A loại bỏ.

Vì vậy nút viền đậm không thể được cắt tỉa một cách an toàn.

d/ Giải thích vì sao cắt tỉa trong cây có nút ngẫu nhiên khác với Alpha-Beta trong cây Minimax.

Cắt tỉa trong cây có chance node phức tạp hơn Alpha-Beta trong Minimax vì chance node không chọn giá trị nhỏ nhất hoặc lớn nhất mà tính giá trị kỳ vọng:

$$V(C) = \sum_i p_i V_i$$

Do đó các nhánh chưa được duyệt vẫn có thể làm thay đổi giá trị của chance node. Trong Minimax, một giá trị đủ tốt tại MAX hoặc đủ xấu tại MIN có thể lập tức tạo ra cận alpha hoặc beta để cắt tỉa. Với chance node, chỉ có thể cắt khi từ các giá trị đã biết và các cận hiện có ta chứng minh được rằng những kết quả chưa xét không thể thay đổi quyết định của tổ tiên. Nếu không có giới hạn đối với giá trị lá, nhiều trường hợp sẽ không thể cắt tỉa an toàn.

Bài 3. Câu hỏi đúng/sai

Xét cây trò chơi có gốc MAX, giá trị Minimax là V_M. Thay mọi nút MIN bằng nút ngẫu nhiên có phân phối xác suất bất kỳ nhưng đã biết, ta nhận cây có giá trị Expectimax là V_E. Với mỗi mệnh đề, chọn Đúng/Sai và giải thích.

a/ Luôn có $VM \leq VE$. ☐ Đúng ☐ Sai

Đúng. Ở cây minimax, tại mỗi nút min ta thấy:

V min = min(V1, V2, …Vk)

Sau khi thay min bằng chance node, giá trị tại node đó là kỳ vọng:

$$V(chance) = \sum_i p_i V_i$$

Với pi ≥ 0, $\sum_i p_i = 1$

Giá trị trung bình có trọng số luôn không nhỏ hơn giá trị nhỏ nhất:

$$\sum_i p_i V_i \geq \min_i V_i$$

Vì vậy tại mỗi nút min được thay bằng chance node, giá trị không thể nhỏ hơn giá trị minimax tương ứng. Max ở phía trên lại luôn chọn giá trị lớn nhất, nên cuối cùng $V_M \leq V_E$

b/ Dùng chính sách Minimax tối ưu ở cây đã thay nút MIN bằng nút ngẫu nhiên sẽ đảm bảo lợi ích kỳ vọng ít nhất là VM. ☐ Đúng ☐ Sai

Đúng. Chính sách minimax tối ưu được xây dựng để đảm bảo max nhận được ít nhất V_M ngay cả trong trường hợp đối thủ min luôn chọn hành động bất lợi nhất cho max.

Khi min được thay bằng chance node, hành động không còn luôn là trường hợp xấu nhất mà được chọn theo một phân phối xác suất.

Giả sử tại 1 node, minimax đảm bảo: $V_i \geq V_M$ cho mọi lựa chọn mà đối thủ có thể thực hiện theo chính sách đó.

Khi lấy kỳ vọng $E[V] = \sum_i p_i V_i$ thì vẫn có $E[V] \geq V_M$

Do đó chính sách minimax tối ưu vẫn đảm bảo được lợi ích kỳ vọng ít nhất bằng V_M.

c/ Dùng chính sách Minimax tối ưu ở cây đã thay nút MIN bằng nút ngẫu nhiên sẽ đảm bảo lợi ích kỳ vọng ít nhất là V_E. ☐ Đúng ☐ Sai

Sai. Chính sách Minimax tối ưu cho trường hợp đối thủ chọn hành động xấu nhất, nhưng chưa chắc tối ưu về kỳ vọng khi các hành động được chọn ngẫu nhiên. Expectimax có thể khai thác phân phối xác suất để chọn một hành động có kỳ vọng cao hơn. Do đó chính sách Minimax chỉ bảo đảm ít nhất V_M, không bảo đảm đạt: V_E

d/ Xét trò chơi một người, kết quả hành động có thể không xác định trước. Người chơi nhận +1 khi thắng và −1 khi thua. Đánh dấu mọi phương pháp có thể phù hợp để mô hình hóa và giải trò chơi theo giả định cần thiết.

☐ Minimax với tiện ích trạng thái cuối +1/−1

☐ Expectimax với tiện ích trạng thái cuối +1/−1

☐ Value Iteration với phần thưởng 0 ở trạng thái trung gian và +1/−1 ở kết quả thắng/thua

☐ Không phương án nào

Giải thích ngắn gọn lựa chọn ở trên và những giả định cần thiết

Không chọn Minimax vì bài toán chỉ có một người chơi và sự không chắc chắn xuất phát từ kết quả ngẫu nhiên chứ không phải một đối thủ có hành vi đối kháng.

Expectimax phù hợp nếu biết xác suất của các kết quả hành động, vì thuật toán chọn hành động có tiện ích kỳ vọng lớn nhất.

Value Iteration phù hợp nếu bài toán có thể mô hình hóa dưới dạng MDP, biết xác suất chuyển trạng thái và trạng thái hiện tại chứa đủ thông tin cần thiết. Có thể đặt phần thưởng bằng 0 ở các trạng thái trung gian, +1 khi thắng và -1 khi thua.

Bài 4. CSP với Constraint Graph và Lan truyền ràng buộc

Có 6 sinh viên A, B, C, D, E, F cần sắp chỗ ngồi. Các cặp có ràng buộc nhị phân: (A,B), (A,C), (A,D), (B,C), (B,E), (C,F).

a/ Vẽ đồ thị ràng buộc tương ứng (mỗi biến là một đỉnh; mỗi ràng buộc nhị phân là một cạnh).

Mỗi biến là 1 đỉnh và mỗi ràng buộc nhị phân là 1 cạnh.

Các cạnh: A – B, A – C, A – D, B – C, B – E, C – F

b/ Sau khi gán giá trị cho A, miền của những biến nào có thể bị thay đổi bởi Forward Checking?

Forward checking chỉ xét các biến chưa gán nối trực tiếp với biến vừa được gán.

Các biến kề với A là: B, C, D. Do đó sau khi gán A, miền có thể thay đổi là B, C, D.

Miền của E và F chưa bị ảnh hưởng vì chúng không có ràng buộc trực tiếp với A.

c/ Sau khi gán A và thực hiện Forward Checking, tiếp tục gán B: miền của những biến nào có thể bị thay đổi khi Forward Checking tại B?

Các biến kề với B là: A, C, E nhưng A đã được gán rồi. Forward checking chỉ xét các biến chưa gán, nên khi gán B ta kiểm tra: C, E.

Do đó C, E có thể bị thu hẹp miền.

d/ Sau khi gán A, nếu duy trì Arc Consistency, miền của những biến nào có thể thay đổi?

Sau khi A được gán, trước hết các biến kề trực tiếp có thể thay đổi: B, C, D.

Nhưng Arc Consistency tiếp tục lan truyền.

Nếu miền B thay đổi nó có thể ảnh hưởng tới: C, E.

Nếu miền C thay đổi nó có thể ảnh hưởng tới: B, F.

Vì vậy ảnh hưởng có thể lan từ A ra toàn bộ thành phần liên thông

A

↓

B, C, D

↓

E, F

Do đó tất cả các biến chưa gán đều có khả năng thay đổi: B,C,D,E,F. Không phải tất cả nhất thiết sẽ thay đổi trong một instance cụ thể, nhưng đều có thể thay đổi do lan truyền ràng buộc.

e/ Sau khi gán A và duy trì Arc Consistency, tiếp tục gán B và duy trì Arc Consistency: miền của những biến nào có thể thay đổi?

Khi gán B, các biến chưa gán kề trực tiếp với B là: C, E nên đây là các miền bị tác động đầu tiên.

Sau đó Arc Consistency tiếp tục lan truyền.

Nếu C thay đổi thì có thể ảnh hưởng tới: F

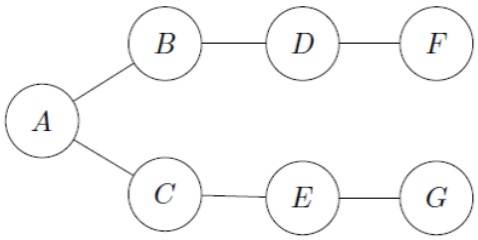
Thông qua các cạnh: B – C, C – F

Do đó: C,E,F có thể thay đổi do phép gán B mới.

Còn D chỉ nối trực tiếp với A: A - D

A đã được cố định và xử lý Arc Consistency từ bước trước. Việc gán B không tạo thêm ràng buộc mới tới D, nên trong quá trình lan truyền mới này D không cần thay đổi.

Xét thuật toán mới: thực hiện Arc Consistency ban đầu và sau mỗi lần đã gán một số chẵn biến (2, 4, 6, ...). Mỗi biến có miền kích thước d. Sử dụng đồ thị ở Hình 3 để phân tích số lần backtrack trong trường hợp xấu nhất cho từng thứ tự gán; nêu rõ cách tính và các giả định.



Hình 3. Đồ thị ràng buộc cho bài toán thứ tự gán biến (nguồn: rec3.pdf, trang 6)

Giả định: CSP ban đầu satisfiable và đã arc-consistent; mỗi phép gán được kiểm tra ngay với các hàng xóm đã gán. Sau mỗi 2 biến được gán, Arc Consistency được chạy toàn cục. Trong trường hợp xấu nhất, nếu hai biến được gán ở hai phía của một đoạn chứa biến chưa gán thì biến thứ hai có thể phải thử tối đa d giá trị, tức d-1 lần backtrack.

f/ Thứ tự A → B → C → D → E → F → G

Với thứ tự A → B → C → D → E → F → G, tập các biến đã gán luôn mở rộng từ phần đã được kết nối của cây. Các biến mới được kiểm tra với hàng xóm đã gán ngay khi chọn giá trị, nên không tạo ra mâu thuẫn ẩn phải chờ lần chạy AC kế tiếp mới phát hiện.

Do đó BTf = 0

g/ Thứ tự G → E → C → A → B → D → F

Với thứ tự G → E → C → A → B → D → F, các biến được gán dần theo một đường liên thông của cây: G – E – C – A – B – D – F. Mỗi biến mới đều được gắn vào phần cây đã gán, nên tính nhất quán có thể được kiểm tra ngay và không có mâu thuẫn ẩn giữa hai phía của một biến chưa gán.

Vì vậy BTg = 0

h/ Thứ tự G → F → E → D → C → B → A

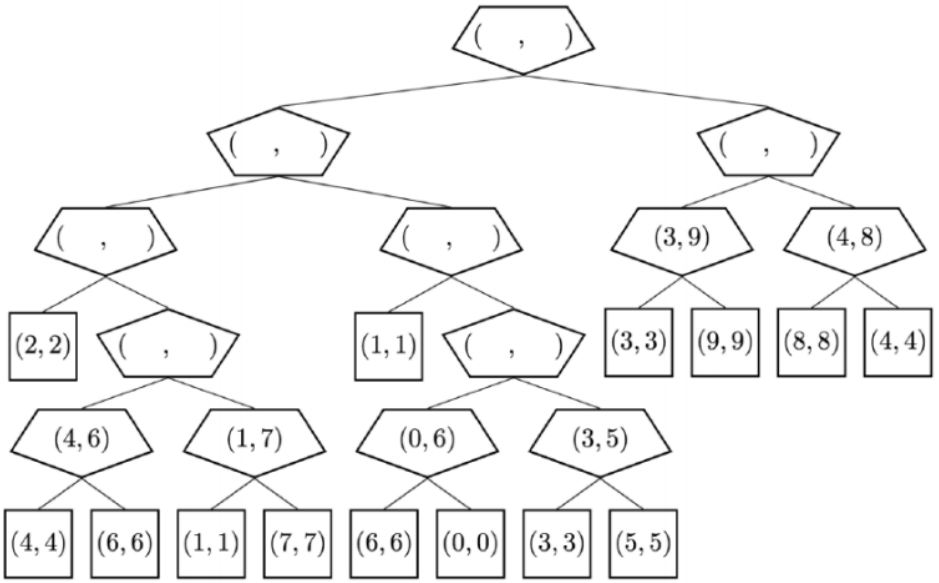
Với thứ tự G → F → E → D → C → B → A, các biến được chọn từ hai đầu của cây tiến dần vào giữa. Các cặp (G,F), (E,D) và (C,B) đều bị ngăn cách bởi ít nhất một biến chưa gán. Vì vậy hai giá trị có thể riêng lẻ hợp lệ nhưng không tương thích với nhau trên toàn bộ đường đi.

Mâu thuẫn chỉ được phát hiện khi Arc Consistency chạy sau mỗi hai phép gán. Với miền kích thước d, trong trường hợp xấu nhất biến thứ hai của mỗi cặp có thể phải thử d giá trị, trong đó d-1 giá trị thất bại.

Vì có ba giai đoạn như vậy nên BTh = 3(d-1)

Bài 5. Cây trò chơi Pacman và Casper

Pacman tìm cách tối đa hóa tiện ích. Casper là một bóng ma thân thiện, thực sự muốn tăng tiện ích cho Pacman nhưng Pacman tưởng Casper cố giảm tiện ích. Casper biết Pacman đang hiểu nhầm. Ở mỗi nút, cặp (u, v) biểu thị lần lượt giá trị mà Pacman tin tưởng và giá trị theo hành vi thực tế của Casper. Tại lá, u = v = tiện ích cuối.



Hình 4. Cây trò chơi Pacman–Casper

a/ Điền tất cả cặp (u, v) còn thiếu trong Hình 4. Giải thích sự khác biệt giữa u và v tại nút Casper.

1/ Nhánh dưới cùng bên trái:

Hai lá: (4,4), (6,6)

Nút cha là Casper.

Theo điều Pacman tin, Casper sẽ chọn nhỏ nhất: u = min(4,6) = 4

Nhưng casper thật sự thân thiện nên chọn lớn nhất: v = max(4,6) = 6

Do đó (4,6)

Nút kế tiếp có 2 lá: (1,1), (7,7) nên u = min(1,7) = 1, v = max (1,7) = 7

Suy ra (1, 7)

Hai nút này là con của 1 nút pacman. Pacman so sánh theo u: 4 > 1 nên pacman chọn nhánh (4,6). Vì vậy nút pacman phía trên là: (4,6)

2/ Nút casper có 2 con (2,2) và (4,6)

Theo niềm tin của pacman: u = min(2,4) = 2

Theo hành vi thật của casper: v = max(2,6) = 6

Do đó (2,6)

3/ Nhánh giữa

Hai lá: (6,6), (0,0)

Nút casper: u = min (6,0) = 0; v = max(6,0) = 6 nên (0,6)

Hai lá tiếp theo: (3,3), (5,5) cho: u = min(3,5) = 3 và v = max(3,5) = 5 nên (3,5)

Nút cha là pacman. So sánh: u1 = 0, u2 = 3. Pacman chọn nhánh thứ hai: (3,5)

4/ Nút casper có hai con (1,1) và (3,5)

Theo niềm tin của pacman: u = min(1,3) = 1

Theo thực tế: v = max(1,5) = 5

Do đó (1,5)

5/ Nút pacman bên trái ở tầng trên

Hai con là: (2,6), (1,5)

Pacman chọn dựa trên u: 2 > 1 nên chọn nhánh (2,6). Vì vậy (2,6)

6/ Nhánh bên phải

Trong hình cho 2 nút casper: (3,9) và (4,8). Nút cha là pacman

Pacman so sánh 3 < 4 nên chọn (4,8)

7/ Nút gốc

Cuối cùng 2 con của nút gốc là: (2,6) và (4,8). Nút gốc là casper

Theo pacman tin, casper chọn u nhỏ hơn u = min (2,4) =2

Nhưng casper thật sự chọn giá trị v lớn hơn: v = max(6,8) = 8

Do đó nút gốc (2,8)

Giải thích sự khác nhau giữa u và v

Tại nút Casper, u và v có thể khác nhau vì Pacman tin Casper là đối thủ và sẽ chọn hành động làm giảm tiện ích, nên u được tính theo giá trị nhỏ nhất. Trong thực tế Casper thân thiện và muốn tăng tiện ích cho Pacman, nên v được tính theo giá trị lớn nhất. Vì vậy tại nút Casper: u = min ui, v = max vi

b/ Theo thứ tự duyệt trái sang phải, đánh dấu các nhánh có thể không cần duyệt khi Casper tính giá trị trò chơi.

Khi duyệt từ trái sang phải, tại nút Pacman có nhánh đầu tiên (4,6), giá trị tin tưởng tốt nhất hiện tại là u=4. Ở nhánh Casper tiếp theo, sau khi gặp lá (1,1), vì Casper được Pacman xem là MIN nên giá trị \(u\) của nhánh này không thể lớn hơn 1. Do 1<4, lá (7,7) không cần duyệt.

Tương tự, tại nút Pacman lớn bên trái, nhánh đầu tiên đã cho u=2. Khi xét nhánh Casper thứ hai và gặp ngay lá (1,1), giá trị u của cả nhánh không thể lớn hơn 1. Vì 1<2, Pacman chắc chắn không chọn nhánh này, nên có thể bỏ qua toàn bộ phần còn lại gồm: (6,6),(0,0),(3,3),(5,5)

Các lá có thể không cần duyệt là: (7,7),(6,6),(0,0),(3,3),(5,5)

c/ Nếu Pacman biết Casper thân thiện thì giá trị toàn bộ cây thay đổi như thế nào?

Nếu Pacman biết Casper thân thiện thì Pacman không còn giả định Casper sẽ giảm tiện ích. Khi đó cả hai đều chọn hành động nhằm tối đa hóa tiện ích của Pacman, nên tại mọi nút:

u = v

Giá trị lớn nhất có thể đạt được trong cây là 9. Vì vậy giá trị tại nút gốc thay đổi từ: (2,8) thành: (9,9). Như vậy việc Pacman hiểu đúng hành vi của Casper giúp Pacman phối hợp với Casper và đạt tiện ích tối ưu cao hơn.

Bài 6. Thuật toán tìm kiếm – Tính đầy đủ và tối ưu

Với mỗi thuật toán sau, xác định điều kiện để được đảm bảo đầy đủ (Complete) và tối ưu (Optimal). Phân biệt tìm kiếm trên cây và trên đồ thị nếu cần.

Thuật toán	Điều kiện đầy đủ (Complete)	Điều kiện tối ưu (Optimal)
BFS	Hệ số phân nhánh hữu hạn, goal ở độ sâu hữu hạn	Các bước có cùng chi phí
DFS	Graph Search trên không gian hữu hạn và tránh lặp; Tree Search không luôn complete	Không đảm bảo
IDS	Hệ số phân nhánh hữu hạn, goal ở độ sâu hữu hạn	Các bước có cùng chi phí
A*	Hệ số phân nhánh hữu hạn, step cost $\geq \varepsilon > 0$	Tree Search: h admissible; Graph Search chuẩn: h consistent

Giải thích vì sao chi phí bước và heuristic ảnh hưởng đến kết luận?

Ảnh hưởng của chi phí bước

BFS và IDS ưu tiên theo độ sâu, không theo chi phí.

Nếu tất cả các bước có cùng cost c thì: $Cost = c \times depth$ nên đường nông nhất cũng là đường rẻ nhất.

Ví dụ: $S \rightarrow G = 100$ chỉ 1 bước. Trong khi: $S \rightarrow A \rightarrow G = 1 + 1 = 2$ có 2 bước.

BFS sẽ thích đường 1 bước, nhưng đường đó không tối ưu về cost.

Ảnh hưởng của heuristic trong A*

A* sử dụng: $f(n) = g(n) + h(n)$

Nếu heuristic admissible: $h(n) \leq h^*(n)$ thì heuristic không đánh giá quá cao chi phí còn lại, giúp A* Tree Search giữ được tính tối ưu.

Nếu heuristic consistent: $h(n) \leq c(n, n') + h(n')$ thì $f(n)$ không giảm dọc theo đường đi, rất quan trọng đối với Graph Search.

Nếu heuristic đánh giá quá cao, A* có thể bỏ qua hoặc trì hoãn nhánh chứa lời giải tối ưu và trả về lời giải có chi phí lớn hơn.

Bài 7. A* khi chi phí một cạnh thay đổi

Sau khi A* graph search tìm được đường đi tối ưu từ trạng thái đầu đến đích, chi phí cạnh X → Y thay đổi từ c thành c + n hoặc c − n, với n > 0. Với từng tình huống, hãy phân tích liệu chi phí tối ưu có thay đổi và cách cập nhật/tìm đường tối ưu mới thay vì nhất thiết chạy lại từ đầu. Khi giảm chi phí, giả sử chi phí mới vẫn hợp lệ theo mô hình tìm kiếm.

a/ Chi phí tăng n; cạnh X → Y thuộc đường tối ưu cũ và đã được khám phá.

Vì cạnh $X \rightarrow Y$ nằm trên đường tối ưu cũ nên khi cost tăng thêm n, chi phí của đường cũ trở thành: $C'_{old} = C^* + n$

Lúc này đường cũ có thể không còn tối ưu, vì một đường khác trước đây đắt hơn C^*nhưng lại rẻ hơn $C^* + n$ có thể trở thành đường tốt nhất.

Do đó chi phí tối ưu mới có thể:

- Tăng
- Hoặc giữ nguyên nếu tồn tại 1 đường khác có chi phí bằng C^*

Cách xử lý là cập nhật cost của cạnh, cập nhật các giá trị g của Y và các hậu duệ phụ thuộc vào đường qua $X \rightarrow Y$, rồi đưa các node bị ảnh hưởng trở lại frontier để tiếp tục tìm kiếm.

b/ Chi phí giảm n; cạnh X → Y thuộc đường tối ưu cũ và đã được khám phá.

Đường tối ưu cũ chứa cạnh này, nên chi phí của chính đường đó giảm: $C'_{old} = C^* - n$

Vì trước khi thay đổi không có đường nào rẻ hơn C^*, và chỉ có một cạnh trên đường tối ưu được giảm giá, nên đường cũ vẫn là một ứng viên tối ưu rất mạnh.

Thực tế, không đường nào không sử dụng cạnh vừa giảm có thể đột nhiên trở nên rẻ hơn trước, vì chi phí của chúng không đổi.

Do đó chi phí tối ưu mới là: $C^* - n$ và đường tối ưu cũ vẫn tối ưu.

c/ Chi phí tăng n; cạnh X → Y không thuộc đường tối ưu cũ nhưng đã được khám phá.

Vì cạnh này không thuộc đường tối ưu cũ nên chi phí của đường tối ưu hiện tại: C^* không thay đổi.

Hơn nữa một cạnh ngoài đường tối ưu lại tăng cost thì các đường sử dụng cạnh đó chỉ trở nên xấu hơn, không thể tạo ra 1 đường mới tốt hơn.

Vì vậy, đường tối ưu cũ vẫn tối ưu và $C^*_{new} = C^*$

d/ Chi phí giảm n; cạnh X → Y không thuộc đường tối ưu cũ nhưng đã được khám phá.

Cạnh này không thuộc đường tối ưu cũ, nhưng khi cost giảm $c' = c - n$ một đường đi qua $X \rightarrow Y$ có thể trở nên rẻ hơn C^*

Do đó chi phí tối ưu mới có thể giữ nguyên hoặc giảm. Ta không thể kết luận ngay.

Vì cạnh đã được khám phá, có thể dùng các giá trị $g(X), g(Y)$ đã biết để thử cập nhật:

$$g_{new}(Y) = \min(g_{old}(Y), g(X) + c - n)$$

Nếu $g(Y)$ giảm, ta cập nhật Y và lan truyền cải thiện xuống các hậu duệ của Y, đưa chúng trở lại frontier nếu cần.

e/ Chi phí tăng n; cạnh X → Y không thuộc đường tối ưu cũ và chưa được khám phá.

Cạnh này:

- không thuộc đường tối ưu cũ;
- chưa được A* khám phá;
- lại còn tăng chi phí.

Do đó đường tối ưu cũ hoàn toàn không bị ảnh hưởng.

Một cạnh chưa được sử dụng mà còn trở nên đắt hơn không thể tạo ra một đường mới tốt hơn. Vì vậy: $C^*_{new} = C^*$và không cần mở rộng thêm chỉ vì thay đổi này.

f/ Chi phí giảm n; cạnh X → Y không thuộc đường tối ưu cũ và chưa được khám phá

Cạnh chưa được khám phá nên A* hiện tại có thể chưa có đủ thông tin về đường đi qua cạnh đó.

Sau khi giảm cost: $c' = c - n$ một đường hoàn toàn mới qua $X \rightarrow Y$ có thể trở nên rẻ hơn đường tối ưu cũ. Do đó: chi phí tối ưu có thể giảm.

Nhưng vì cạnh chưa được khám phá nên không thể chỉ cập nhật một $g(Y)$ đã có sẵn một cách chắc chắn.

Ta cần tiếp tục hoặc khởi động lại phần tìm kiếm liên quan, mở rộng những node có khả năng dẫn tới X và cạnh $X \rightarrow Y$.

Nếu hệ thống còn giữ frontier/open list từ lần chạy trước, có thể tái sử dụng chúng; nếu không giữ đủ thông tin cần thiết thì có thể phải chạy lại A* từ trạng thái đầu.

Bảng tổng hợp:

Trường hợp	Chi phí tối ưu có thể thay đổi?	Cách xử lý
a/ Tăng, cạnh thuộc đường tối ưu, đã khám phá	Có thể tăng	Cập nhật g của Y và hậu duệ, xét lại đường thay thế
b/ Giảm, cạnh thuộc đường tối ưu, đã khám phá	Giảm đúng n	Giữ đường cũ, cập nhật cost
c/ Tăng, cạnh không thuộc đường tối ưu, đã khám phá	Không	Giữ nguyên đường tối ưu
d/ Giảm, cạnh không thuộc đường tối ưu, đã khám phá	Có thể giảm	Relax lại cạnh, cập nhật Y và các hậu duệ
e/ Tăng, cạnh không thuộc đường tối ưu, chưa khám phá	Không	Không cần tìm lại
f/ Giảm, cạnh không thuộc đường tối ưu, chưa khám phá	Có thể giảm	Tiếp tục/mở lại tìm kiếm vùng liên quan; nếu thiếu dữ liệu thì có thể chạy lại A*

Bài 8. Tác tử thông minh và môi trường

Xét ba kiểu tác tử, trong đó các hàm f_A, f_B, f_C có thể ngẫu nhiên nhưng không lưu trạng thái nội bộ:

• A: act(percept) → f_A()

• B: act(percept) → f_B(percept)

• C: lưu danh sách percepts; mỗi lần act(percept) thêm percept rồi trả về f_C(percepts)

Với mỗi đặc tính môi trường sau, hãy liệt kê các lớp tác tử có thể tồn tại một hàm f để hành động hợp lý hoàn hảo trong mọi môi trường thuộc lớp đó. Giải thích ngắn gọn.

a/ Quan sát đầy đủ; xác định; rời rạc; một tác tử; tĩnh.

Trong môi trường **quan sát đầy đủ**, percept hiện tại cung cấp đầy đủ thông tin cần thiết về trạng thái hiện tại. Vì môi trường **xác định**, một hành động trong một trạng thái xác định sẽ cho một kết quả xác định. Do đó tác tử chỉ cần biết trạng thái hiện tại để lựa chọn hành động hợp lý.

Tác tử A

A không sử dụng percept: $f_A()$ nên không thể thay đổi hành động theo trạng thái môi trường.

Có thể có 2 trạng thái: s_1, s_2 trong đó hành động tối ưu khác nhau. A không phân biệt được hai trạng thái này. Vì vậy A không được đảm bảo hành động hợp lý hoàn hảo trong mọi môi trường thuộc lớp này.

Tác tử B

B sử dụng percept hiện tại: $f_B(percept)$

Do môi trường quan sát đầy đủ, percept hiện tại đã cho đủ thông tin về trạng thái. Vì vậy có thể tồn tại f_B ánh xạ mỗi trạng thái vào hành động tối ưu => B phù hợp

Tác tử C

C có toàn bộ lịch sử percept nên ít nhất cũng có thông tin mà B có.

Trong môi trường quan sát đầy đủ, C hoàn toàn có thể chỉ sử dụng percept cuối cùng nếu lịch sử không cần thiết => C phù hợp

Kết luận: B, C

b/ Quan sát một phần; xác định; rời rạc; một tác tử; tĩnh

Trong môi trường **quan sát một phần**, percept hiện tại có thể không cung cấp đủ thông tin để biết chính xác trạng thái. Hai trạng thái khác nhau có thể cho cùng một percept: $s_1 \rightarrow p$, $s_2 \rightarrow p$ nhưng hành động hợp lý tại hai trạng thái có thể khác nhau.

Tác tử A

A hoàn toàn không dùng percept nên càng không thể sử dụng thông tin quan sát để lựa chọn hành động => A không phù hợp

Tác tử B

B chỉ nhìn percept hiện tại. Nếu hai trạng thái khác nhau tạo ra cùng percept nhưng yêu cầu hành động khác nhau thì B không thể phân biệt được chúng.

Ví dụ: $p_t = p$ có thể tương ứng với cả s_1 và s_2

B luôn phải thực hiện: $f_B(p)$ giống nhau trong cả hai trường hợp.

Do đó B không đủ mạnh cho môi trường quan sát 1 phần => B không phù hợp

Tác tử C

C sử dụng toàn bộ chuỗi $(p_1, p_2, \dots p_n)$

Lịch sử quan sát có thể cung cấp thông tin để suy luận về trạng thái hiện tại, ngay khi cả percept hiện tại riêng lẻ chưa đủ. Do đó C có thể xây dựng quyết định dựa trên thông tin tích lũy từ quá khứ => C phù hợp

Kết luận: C

c/ Quan sát một phần; ngẫu nhiên; rời rạc; một tác tử; tĩnh.

Trường hợp này khó hơn vì có cả:

- **partial observability**: không thấy đầy đủ trạng thái;
- **stochasticity**: cùng một hành động có thể dẫn tới các kết quả khác nhau.

Tác tử A

A không dùng percept hay lịch sử: $f_A()$ nên không thể điều chỉnh hành động theo thông tin thu được => A không phù hợp

Tác tử B

B chỉ dựa vào percept hiện tại. Trong môi trường quan sát một phần và ngẫu nhiên, cùng percept hiện tại có thể xuất hiện sau nhiều lịch sử khác nhau, và các lịch sử đó có thể dẫn đến các mức tin tưởng khác nhau về trạng thái thật.

Ví dụ: $p_t = p$ nhưng sau khi lịch sử thứ nhất ta tin trạng thái s_1 có xác suất cao, trong khi sau lịch sử thứ hai ta tin s_2 có xác suất cao.

Do đó hành động tối ưu có thể phụ thuộc vào lịch sử. B không có thông tin này => B không phù hợp.

Tác tử C

C lưu: $p_1, p_2, \dots p_n$ nên có thể dùng toàn bộ lịch sử quan sát để duy trì thông tin về các trạng thái có thể xảy ra và lựa chọn hành động tối ưu theo kỳ vọng.

Nói cách khác, lịch sử percept có thể được dùng để hình thành một dạng belief state về trạng thái môi trường. Do đó trong ba kiến trúc được đề bài cung cấp, C là kiến trúc đủ khả năng biểu diễn một chính sách phụ thuộc lịch sử => C phù hợp

Kết luận: C

d/ Đúng hay sai: Có tồn tại một môi trường mà mọi tác tử đều hành động hợp lý. Nêu ví dụ hoặc phản biện.

Đúng. Có thể tồn tại một môi trường trong đó mọi hành động đều cho cùng một mức tiện ích. Ví dụ, nếu môi trường chỉ có một trạng thái và mọi hành động đều cho phần thưởng

bằng 1 thì không có hành động nào tốt hơn hành động khác. Do đó bất kể tác tử A, B hay C chọn hành động nào thì hành động đó vẫn là hợp lý.

Vì vậy có thể tồn tại môi trường mà mọi tác tử đều hành động hợp lý.